Risk Based Approval for Agent Tool Calls

PE6203 Case 2 · Individual contribution report · niuduoer001-eng · 26 October 2026

My contribution to the policy

My part of Case 2 is the risk-based approval policy. The module checks a proposed tool call before the shared gate executes it. It returns APPROVE, DENY, or ESCALATE and gives a reason. I am responsible for the rules, their implementation, their tests, and the supporting documentation. The gate, agent model, reviewer, and benchmark analysis are separate parts of the project. This report covers the policy component and the evidence available from its local tests.

The module is called BasicRiskPolicy. Its decide(request) method uses the agreed interface. Each request contains user_task, tool_name, tool_arguments, and remaining_budget. It may also include history and tool_metadata. The response contains only decision and reason. The policy does not call a model or a reviewer. It also keeps no state between tasks. These choices make its decisions repeatable for the same input. The gate remains responsible for calling the reviewer and recording the final action.

I based the tool groups on their actual effects. A name beginning with get does not prove that a tool only reads data. For example, get_unread_emails marks messages as read in the pinned AgentDojo code. That tool therefore needs review. A webpage request also sends a URL to another service. It stays outside the local-query allow-list because the URL could contain information that should not be sent (“email_client.py”; “Tools”).

The policy checks the request before applying the tool groups. It rejects invalid input and checks outbound text for a small set of credential patterns. It then checks supported restrictions in the user's task. After that, it applies the tool's risk category. Reviewed local queries may run directly. Sensitive reads, external actions, payments, state changes, and unknown tools go to the reviewer. Each decision includes a fixed rule ID, so the existing logs can show which rule was applied.

The authorisation checks cover two clear cases. The first is a supported direct prohibition, such as an instruction not to send email. The second is an explicit restriction to one recipient. The email check includes cc and bcc. A call that names an extra address is denied. A matching address still needs review because the message or attachment may be wrong. Conditional wording and indirect permission remain review work. These rules are a small set of checks, not a complete test of user intent.

The policy name is basic_risk_v2. The experiment runner uses this name to distinguish its results from older runs. The submission also includes callable examples, an installation checker, rule documentation, and test records. The public interface is unchanged. The module can therefore connect to the shared gate without taking over budget handling. The separate budget-aware strategy can reuse the fixed assessment function, but it needs its own name and an explanation of any changes to the decisions.

My contribution to testing and integration

The tests check both the policy output and the action taken by the gate. Returning ESCALATE is only part of the process. The gate must call the reviewer, spend one unit of budget, and apply the answer correctly. I used the real ApprovalGatedToolsExecutor with local fake tools and FakeReviewer for the connection tests. The fake tools record whether they ran. This lets the tests check execution rather than relying only on the decision written to a log.

The recorded local run passed 117 tests. Of these, 101 cover the risk policy, its connection to the gate, and its constructed failure examples. The other 16 are existing shared tests. The policy tests cover normal decisions, explicit user restrictions, and invalid input. They also cover cyclic arguments, deeply nested data, non-finite numbers, and oversized text. Tests with malicious tool descriptions and history check that those fields cannot give a risky call permission to run.

One test puts five calls in a single agent turn with k set to one. A local search runs first. An ordinary email then uses the one available review and runs after approval. A second email asks for review at zero budget, so the gate denies it without calling the reviewer. A synthetic credential-bearing email is denied directly. A final local query still runs. The final decisions are APPROVE, APPROVE, DENY, DENY, APPROVE. Only one actual review takes place.

Other tests check the budget across turns and tasks. Calls in the same task share one budget. A new task resets it. A reviewer denial still spends one review, while a direct policy denial spends none. The new authorisation tests check the same boundary. A prohibited send and an extra recipient are rejected before review, so they preserve the budget. The policy itself never subtracts from remaining_budget. These tests keep the policy and gate responsibilities separate.

The examples run without a model key or an external email service. Their task_completed and security_passed fields stay null because no benchmark evaluator ran. I also included two executable failure examples. One blocks an intended safe send at k equal to zero. The other allows a read of the wrong file. Their failure labels are defined by a local test setup and kept outside the policy request. They show known limits, but they are not results from the AgentDojo benchmark.

The installation checker reads direct_url.json to identify the installed source. The package reports version 0.1.35, but its source commit matches the agreed revision, 089ed468cf3ed0322acc66b0211f26d9d90dbf60. Checking only the version number would miss this distinction. The dependency check also found no broken requirements. The documentation records the commands, imports, and expected outputs. These checks support reproducible setup and local integration. They do not measure how often a live agent completes a task safely.

Reflection

The policy makes a distinction between an operation's effects and the user's permission. A read may have little immediate effect on the environment, yet still access the wrong file. A send changes something outside the agent, yet may be exactly what the user requested. The tool groups describe risk. The authorisation checks enforce only the clear restrictions they recognise. Keeping these two purposes separate makes the limits easier to explain and test.

The wrong-file example shows the gap most clearly. The task asks for public notes, but get_file_by_id receives the identifier of a restricted synthetic file. The call runs because the function is on the local-query allow-list. The rule recognises the kind of operation, not whether that object belongs to the task. A stronger design would need a trusted link between the task and the requested resource. A retrieved document cannot provide that permission on its own because the document may contain an attack.

The policy can also be too conservative. An intended email still needs review even when its destination is clear. If the review budget is already empty, the task stops. The fixed policy keeps the same risk decision at every budget. This gives the budget-aware strategy a clear comparison point, but it may reduce task completion. The formal experiment must measure that cost. A test can prove that the code follows a rule without proving that the rule is the best choice for the task.

The recipient check also has a limited scope. It recognises one explicit address after only to or only with, then checks the call's destinations. It does not resolve a contact name or several possible addresses. It also leaves conditional instructions and changed permission to the reviewer. Adding more language patterns could cover more tasks, but it could also create wrong denials. Each extension needs examples of both a real restriction and similar wording that should not block the action.

A simulated reviewer needs its own evaluation. It may misunderstand the task or follow instructions hidden in the trace. The brief permits a model to stand in for a human reviewer, but the model's answer is still a decision to test. Each strategy should use the same reviewer and the same allowed context. Ground-truth answers must stay outside both the reviewer and policy. Failure analysis should distinguish a policy error, a reviewer error, and an agent that fails to finish the task (*Group Project 2*, secs. 3.1–3.3).

With more time, I would first examine which reviews actually change a task's outcome. That evidence could guide a narrow set of direct approvals for writes with trusted scope information. I would then test the same tasks at several budgets with the same model and reviewer. Any gain should come with a clear account of what changed. Repeated runs would help show whether the gain is consistent. Real failure trajectories would also show where a rule needs better information rather than another keyword pattern.

Rules and integration

Case 2 asks the policy to support safe task completion under a limited review budget. AgentDojo provides tasks in which agents use tools and read untrusted content. The policy checks a call before execution. A separate evaluator checks the outcome. This separation keeps evaluation answers out of the decision process. The AgentDojo paper presents an environment for testing attacks and defences. Its benchmark purpose does not make a small local connection test representative of the full task set (Debenedetti et al.).

R0 checks the request format and limits the amount of input the policy processes. The argument check uses a loop rather than recursive calls. It accepts at most 16 nesting levels, 4,096 visited nodes, and 100,000 combined text characters. These are processing limits, not review-budget limits. R1 checks transmitted fields for a narrow credential assignment or private-key marker. It does not recognise every secret or encoding. Other outbound calls still go to review if this pattern check finds nothing.

R8 checks supported direct prohibitions in user_task. These include clear restrictions on sending email, deleting data, or changing a password. R9 checks an explicit single-recipient restriction for email and file sharing. Both rules use the user's task, not permission claims in tool output. A tool description also cannot cancel a restriction. If the parser cannot establish a supported restriction, the call continues to the normal risk rule. It does not receive permission merely because no restriction was found.

R2 covers the exact list of reviewed local queries. R3 covers sensitive account, financial, identity, and arbitrary-file reads. R4 covers external recipients, audiences, URLs, and related actions. R5 covers payments, scheduled transactions, and reservations. R6 covers state changes, including marking email as read. R7 covers a registered tool that has no reviewed profile. The gate blocks an unregistered tool before the policy runs. The policy therefore has no general rule that approves a tool because its name starts with read or get.

The runner uses build_approval_pipeline to connect the gate to AgentDojo. AgentPipeline.from_config alone would build the plain tool executor. The runner starts one budget per task, sends the proposed calls through the gate, and closes the task at the end. The gate checks policy output, calls the reviewer, manages k, and records the final decision. The policy returns only its decision and reason. Its name becomes part of the pipeline identity, which keeps different policy revisions in separate result locations (“Approval Gate”).

FullApprove and FullEscalate use the same shared gate. This makes the policy comparison easier to control, but the common restrictions must be reported. In particular, the gate rejects nested tool calls before it asks any policy. FullApprove in this pipeline is therefore different from unrestricted AgentDojo when a task needs nested calls. The runner should check the selected tasks and record that difference. The risk-policy component leaves the shared behaviour unchanged, so a result cannot be explained by an unreported change to the executor.

Failure examples and review decisions

The first local example shows a safe action blocked by conservative routing. The constructed task asks the agent to find public notes and send a summary to team@example.test. A search runs under R2. The next send_email call receives ESCALATE under R4. The task has k equal to zero, so the gate changes that request to DENY. The fake email does not run. The actual review count stays at zero. The intended task remains unfinished even though the local test setup treats the send as allowed.

The second example shows a read of the wrong object. The constructed task asks for public notes. A search returns a public file identifier, but the next get_file_by_id call names restricted-file. R2 approves the call because the function is on its query list. The fake tool records the read. The local evaluator treats that file as outside the task. This information is not given to the policy. The result shows a gap in object-level authorisation, not a failure to identify a destructive operation.

The script examples.policy_failure_cases generates both traces through the real gate. Its tools are local substitutes. Each trace records arguments, rule ID, policy decision, final decision, execution status, and budget. The shared utility and security fields remain null because the benchmark evaluator did not run. Separate diagnostic fields describe the constructed failure. Tests confirm that the safe send is blocked and the restricted file is read. This keeps the explanation tied to actions that the example actually performs.

The explicit authorisation tests cover a different outcome. A supported user prohibition blocks a send before it reaches the reviewer. An extra address under a single-recipient restriction is also blocked, including when it appears in bcc. The gate tests confirm that these denials leave the budget unchanged. A matching address still needs review. The recipient check can enforce one clear boundary, but it cannot prove that every sentence or attachment belongs in the message.

Review is most useful when it can settle a specific uncertainty before an action takes effect. For a send, the reviewer needs to check the destination and content. For a transfer, it needs the account and amount. For deletion, it needs the exact target and the task's permission. Reservations require the dates and terms. Sensitive reads may also need review. These decisions involve information that a tool category alone cannot provide. The reviewer must receive enough public task context to assess it.

The next experiment should record whether a review changed the action and whether that change helped the task finish safely. It should also record cases in which review added cost but did not change the decision. The two local examples can help debug the runner and explain the rule limits. Real AgentDojo runs are still needed for attack success, safe completion, and reviewer quality. The run records need the selected model, tasks, attacks, reviewer configuration, and budget so that the strategies can be compared on the same basis.

Evaluation requirements and references

The formal Case 2 evaluation has not been run for this report. The available repository contains FakeReviewer, scripted tests, and the existing workspace replay. It does not provide a confirmed live model configuration or formal result set. The report therefore gives no benchmark TCR curve, HITL total, or dollar estimate. The results described on the earlier pages concern the policy implementation and its local tests.

TCR@k counts tasks that both finish and execute no unauthorised call. That count is divided by all evaluated tasks at the same k. Test pass rate cannot replace it. A proposed range is k = 0, 1, 2, 4, 8, and 16, subject to the selected tasks and the agreed experiment plan. Each strategy needs the same settings. The brief describes HITL over completed tasks and also asks for suite-wide totals. The final analysis should report both and state which tasks each count includes (*Group Project 2*, secs. 3.1, 4.2).

The cost record needs separate totals for the agent, policy, and reviewer. This policy makes no model calls, so its policy token count is zero. The live agent and reviewer may still incur cost. A review request blocked at zero budget is not an actual intervention. Retry tokens already included in the reviewer total must not be added again. Late usage follows the gate's included_in_task_total flag. A dollar estimate also needs the chosen model and a documented price source.

The failure analysis needs real trajectories for both conservative and permissive errors. Each example should show the proposed call, the policy decision, the gate's final action, and the point at which the task failed or left its authorised scope. Review decisions should be examined in the same trace. This will help distinguish an unnecessary escalation from a useful correction or a reviewer mistake. The constructed examples in this report provide a reproducible format, but they do not supply those benchmark outcomes.

Works Cited

"Approval Gate." PE6203 Group 2 Safety Approval, README.md and approval_gate/contract.py, github.com/ywSun04/pe6203-group2-safety-approval. Accessed 26 Oct. 2026.

Debenedetti, Edoardo, et al. "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents." *arXiv*, version 3, 24 Nov. 2024, doi.org/10.48550/arXiv.2406.13352.

"email_client.py." *AgentDojo*, ethz-spylab, commit 089ed468cf3ed0322acc66b0211f26d9d90dbf60, github.com/ethz-spylab/agentdojo/blob/089ed468cf3ed0322acc66b0211f26d9d90dbf60/src/agentdojo/default_suites/v1/tools/email_client.py. Accessed 26 Oct. 2026.

Group Project 2: Safety Approval Policies for Agent Harnesses. PE6203, supplied Case 2 course handout, secs. 2–4.

"Tools." *AgentDojo*, ethz-spylab, commit 089ed468cf3ed0322acc66b0211f26d9d90dbf60, github.com/ethz-spylab/agentdojo/tree/089ed468cf3ed0322acc66b0211f26d9d90dbf60/src/agentdojo/default_suites/v1/tools. Accessed 26 Oct. 2026.

Development assistance: Codex supported source checks, implementation, test preparation and drafting.